CSS Revision Notes

Selectors, box model, positioning, flexbox, grid and Bootstrap, with code you can copy and try

Box colour	What it means
CLEARER	Your note was right but a bit hard to follow; this explains it better.
CORRECTION	Something in the handwritten notes was not accurate. Fix this before exams or interviews.
BONUS	Small extra that is not in your notes but is needed in real projects.

Contents: 1 Adding CSS | 2 Selectors | 3 Colours and units | 4 Box model | 5 div and display | 6 Cascading and specificity | 7 Positioning | 8 Float and clear | 9 Responsive design and media queries | 10 Flexbox | 11 Grid | 12 Flexbox vs Grid | 13 Bootstrap | 14 Quick revision and common mistakes

1. Adding CSS to HTML

CSS rule format: `selector { property: value; }`. The selector says *which* elements, the property says *what* to change, the value says *how*.

Type	Where it lives	Use when
Inline	`style="..."` attribute on one tag	One-off quick change on a single element
Internal	`<style>` tag, normally inside `<head>`	A single HTML page
External	Separate `.css` file linked with `<link>`	Multi-page websites (best practice)

```
<!-- Inline -->
<h1 style="color: red;">Hello</h1>

<!-- Internal (inside <head>) -->
<style>
  h1 { color: red; }
</style>

<!-- External: link the file inside <head> -->
<head>
  <link rel="stylesheet" href="./style.css">   <!-- rel = relationship, href = file path -->
</head>
```

CLEARER: In your notes, "internal can be applied anywhere in the HTML doc". Technically a `<style>` tag works in the body too, but always put it in `<head>`. It applies to that one page only.

2. Selectors

2.1 The five basic selectors

```
/* 1. Element selector: targets every <h1> */
h1 { color: navy; }

/* 2. Class selector: dot + name. Can be reused on MANY elements */
.card { border: 1px solid gray; }          /* <div class="card"> ... <p class="card"> */

/* 3. ID selector: hash + name. Use on ONE element per page (must be unique) */
#main { width: 80%; }                      /* <div id="main"> */

/* 4. Attribute selector: matches elements having an attribute (or a specific value) */
p[draggable] { color: red; }               /* any <p draggable="..."> */
p[draggable="false"] { color: red; }       /* only when the value is "false" */
input[type="text"] { border: 1px solid blue; }

/* 5. Universal selector: every element on the page */
* { margin: 0; padding: 0; }
```

Class (`.name`)	ID (`#name`)
Can be applied to many elements	Only one element in a HTML file; must be unique
One element can have several classes: `class="card big"`	One element has one id
Use for styling groups of things	Use for one-off things / JS hooks / link anchors

2.2 Combining selectors (combinators)

Name	Syntax	Meaning
Group	`h1, h2, p { }`	Same rule for all listed selectors (comma = OR)
Child	`div > p { }`	Only **direct** children (one level), not grandchildren
Descendant	`div p { }`	Any `p` inside `div`, at any depth (space = descendant)
Chaining	`h1.heading#title { }`	ONE element that has all these features (no space)
Combined	`.menu li.active { }`	Mix of the above: `li` with class `active` inside `.menu`

```
<div class="menu">
  <p>Direct child of div</p>
  <section>
    <p>Grandchild: NOT matched by "div > p", but matched by "div p"</p>
  </section>
</div>
```

```
h1, h2, p        { color: blueviolet; }   /* group  */
div > p          { color: firebrick; }    /* child: only the first <p> above */
div p            { color: blue; }         /* descendant: both <p> above */
p.note           { color: seagreen; }     /* <p class="note"> (chained, no space) */
.card .title     { font-size: 1.2rem; }   /* .title INSIDE .card (space) */
.card.title      { font-size: 1.2rem; }   /* ONE element with both classes (no space) */
```

CORRECTION: Your notes say a chaining selector "always starts with the element". That is only true when you use an element name (`h1.heading`). `.card.featured` is a valid chain with no element. The real rule: no space between parts = same element. Space = descendant.

3. Colours and units

```
body {
  background-color: lightyellow;   /* background-color = the background */
  color: #333333;                  /* color = the TEXT colour */
}
```

Format	Example	Note
Name	`red`, `navy`	About 140 named colours
Hex	`#FF0000`	#RRGGBB (red, green, blue)
RGB	`rgb(255, 0, 0)`	Each value 0 to 255
Hex with transparency	`#FF000080`	Last 2 digits = alpha (opacity). `FF` solid, `80` about 50%, `00` invisible

Units

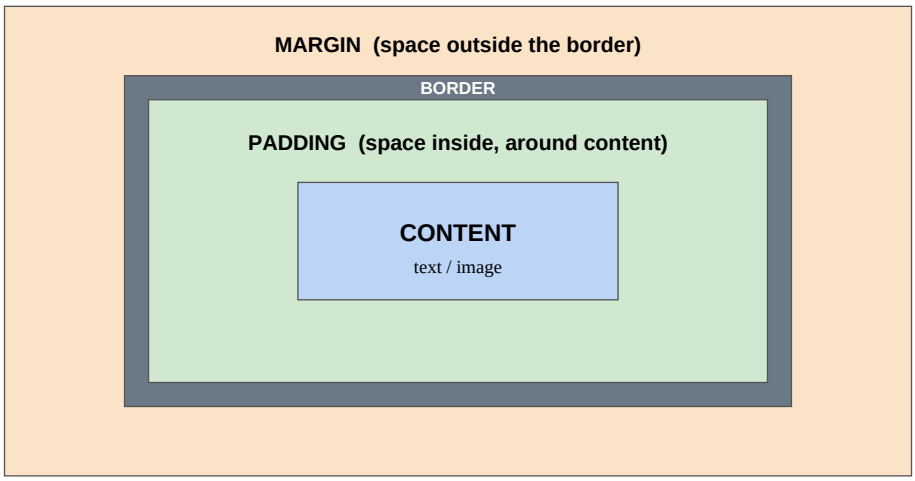
Unit	Relative to	Example
`px`	Fixed pixels	`width: 200px`
`em`	Font size of the **parent** (for `font-size`)	parent 20px, `2em` = 40px
`rem`	Font size of the **root** (`html`), default 16px	`2rem` = 32px, always
`%`	Parent's size	`width: 50%`
`vh` / `vw`	1% of viewport height / width	`height: 100vh` = full browser height

```
html     { font-size: 16px; }
.box     { font-size: 20px; }
.box h1  { font-size: 2em;  }   /* 2 x 20px (parent .box) = 40px */
.box h2  { font-size: 2rem; }   /* 2 x 16px (root html)  = 32px, nesting never changes it */
```

CLEARER: Why em is risky: it **compounds** when nested. A `1.5em` list inside a `1.5em` list becomes 2.25 times the base size. `rem` avoids this because it always looks at the root. Rule of thumb: use `rem` for font sizes and spacing, `px` for borders, `%`/`vh`/`vw` for layout.

4. Box model

Every element is a box made of four layers, from inside to outside: **content, padding, border, margin**.



- `padding`: space between the content and the border (the background colour fills it).
- `margin`: space between the border and other elements. It adds gaps between elements that padding cannot create.
- `border`: `border: thickness style color;` e.g. `border: 2px solid black;`

```
.box {
  border: 2px solid black;
  padding: 20px;                /* all four sides */
  margin: 10px 20px;            /* top+bottom 10px, left+right 20px */
}
/* Four values go CLOCKWISE from the top: top, right, bottom, left */
.a { padding: 10px 20px 30px 40px; }
.b { margin: 0 auto; }          /* auto left/right centres a block that has a width */
```

BONUS: By default `width` only sets the *content* width, so padding and border make the box bigger than you expect. Add `* { box-sizing: border-box; }` and `width` will include padding and border. Almost every project does this.

5. div and the display property

`<div></div>` is the "Content Division" element: an invisible container used to group parts of a page. It shows nothing until you style it (colour, border, size, layout).

display	Behaviour	Width / height?
`block`	Starts on a new line and takes the full width	Yes
`inline`	Sits in a line with text; only as wide as its content	No, ignored
`inline-block`	Sits in a line like inline, but you can set width/height	Yes
`none`	Element disappears completely from the page	n/a

```
<span class="tag">HTML</span> <span class="tag">CSS</span> <span class="tag">JS</span>

.tag {
  display: inline-block;   /* stay on one line AND allow a size */
  width: 80px;
  height: 30px;
  background: lightblue;
}
```

- Block by default: `div`, `p`, `h1` to `h6`, `ul`, `li`. Inline by default: `span`, `a`, `strong`.

CORRECTION: Your note lists `<p>` and `<img/>` together with `span` as inline. `p` is actually **block**. `img` is inline-level but, being an image, you CAN set its width and height. Also, `display: none` removes the element and its space; `visibility: hidden` hides it but keeps the empty space.

6. Cascading and specificity

When two rules try to style the same property of the same element, the browser needs a winner. That is the "cascade".

The three questions the browser asks, in this order

Step	Check	Winner
1. Importance	Does a rule have `!important`?	The `!important` rule
2. Specificity	How specific is the selector?	More specific selector
3. Order (position)	Still tied?	The rule written **last**

Specificity ranking (high to low)

Level	Selector type	Weight
1 (strongest)	Inline `style=""`	1000
2	ID `#id`	100
3	Class `.class`, attribute `[x]`, pseudo-class `:hover`	10
4 (weakest)	Element `h1`, `li`	1

```
<li id="first-id" class="first-class" draggable="true">Item</li>

li               { color: blue;   }   /* weight 1   */
.first-class     { color: red;    }   /* weight 10  */
li[draggable]    { color: purple; }   /* weight 11  (element 1 + attribute 10) */
#first-id        { color: orange; }   /* weight 100 -> WINS, shown in orange */

/* Position (order): same specificity, the last one wins */
li { color: red; }
li { color: blue; }                   /* blue wins */

/* Importance: beats everything normal */
li { color: green !important; }       /* green wins even over #first-id */
```

CORRECTION: Your notes rank **Importance > Type > Specificity > Position** and say external is level 3 (weakest) and internal is level 2. That is not how CSS works. A `<style>` block and a linked `.css` file have **equal weight**. Between them it is decided by specificity first, then by which one comes later in the HTML. Only inline styles always beat them, and that is because inline is the highest *specificity* level. Correct order to remember: **!important > specificity (inline > id > class > element) > order**.

CLEARER: A class and an attribute selector have the same weight (10), so between those two the later rule wins. That is why `li[draggable]` (written later) beats `.first-class` in the sample when there is no id.

BONUS: Practical use: to override Bootstrap, put your own CSS link **after** the Bootstrap link, and make your selector at least as specific. Avoid `!important` unless nothing else works, since it makes later changes painful.

7. Positioning

`position`	Behaviour	Moves with `top/left/right/bottom`?
`static`	Default. Normal document flow.	No, these properties are ignored
`relative`	Stays in normal flow, but can be shifted from its original spot. Its old space stays reserved.	Yes, offset from its own original position
`absolute`	Removed from normal flow. Placed relative to the **nearest positioned ancestor**.	Yes, offset from that ancestor
`fixed`	Placed relative to the browser window (viewport). Stays put when you scroll.	Yes, offset from the viewport
`sticky`	Mix of relative and fixed: scrolls normally, then sticks once it reaches a set offset.	Yes, threshold like `top: 0`

"Positioned" means any `position` other than `static`. Elements nested inside a div are its **descendants**.

Relative

```
.box { position: relative; left: 150px; }   /* pushes the box 150px to the RIGHT of where it was */
```

CLEARER: `left: 150px` means "move away from the left edge by 150px", so the box shifts right. `top: 20px` shifts it down. Use negative values for the opposite direction.

Absolute (and why the parent is made relative)

```
<div class="profile">Profile <span class="badge">3</span></div>

.profile { position: relative; }       /* anchor: this becomes the reference point */
.badge   {
  position: absolute;                  /* leaves normal flow */
  top: -10px;                          /* measured from .profile's top edge   */
  right: -10px;                        /* measured from .profile's right edge */
}
```

CLEARER: The browser walks up the ancestors until it finds one with `position` that is not `static`. If it finds none, it uses the top-left of the web page. That is the "tricky thing" in your notes: if you forget `position: relative` on the parent, the badge flies to the corner of the whole page. So the pattern is always **parent relative, child absolute**.

z-index, fixed and sticky

```
.modal  { position: absolute; z-index: 10; }   /* higher number = closer to the viewer (on top) */
.navbar { position: fixed; top: 0; left: 0; width: 100%; }   /* stays at top while scrolling */
th      { position: sticky; top: 0; background: white; }     /* table header sticks at top */
```

CLEARER: `z-index` controls which element is on top along the z-axis (towards you). It works on positioned elements (and on flex/grid items). Fixed elements ignore ancestors and use the viewport. Sticky needs a threshold (`top`, etc.) or it never sticks.

8. Float and clear

- History: layouts were first made with tables, then `display`, then `position`, then `float` + `clear`, and now **flexbox and grid**.
- `float` pushes an element left/right so text wraps around it. Use it **only** to wrap text around an image, never for page layout.
- `clear` stops an element from sitting beside a floated element, so it drops below it.

```
<img src="dog.jpg"> <p>Long paragraph that wraps around the image ...</p>
<footer>Footer</footer>

img    { float: left; margin-right: 12px; }
footer { clear: left; }       /* footer goes below the image; use "both" to clear left and right */
```

9. Making a website responsive

Four tools: (1) **media queries**, (2) **CSS grid**, (3) **flexbox**, (4) an external framework such as **Bootstrap**.

BONUS: Phones need this tag in `<head>`, otherwise they pretend to be a wide desktop screen and your media queries do not behave: `<meta name="viewport" content="width=device-width, initial-scale=1">`

Media queries

```
/* max-width: applies to screens 600px wide OR LESS (phones) */
@media (max-width: 600px) {
  h1 { font-size: 15px; }       /* overrides the normal h1 size on small screens */
}

/* min-width: applies to screens 600px wide OR MORE (tablets, laptops) */
@media (min-width: 600px) {
  .container { width: 90%; }
}

/* between two sizes: combine with "and" */
@media (min-width: 600px) and (max-width: 900px) {
  /* styles for 600px to 900px */
}

/* orientation */
@media (orientation: landscape) { /* wide screens / rotated phones */ }

/* print: only used when the page is printed */
@media print { nav, footer { display: none; } }
```

CLEARER: Memory trick: **max-width = "up to"** (targets smaller screens), **min-width = "from"** (targets bigger screens). The word `screen` in `@media screen and (...)` is optional because screens are the default; the alternative type is `print`. Your notes say "not really necessary", which is correct.

10. Flexbox (one-dimensional layout)

Flexbox lines items up along **one** direction (a row or a column) and makes it easy to align and space them, both horizontally and vertically. Setup: wrap items in a container and set `display: flex` on the **container**.

```
<div class="flex-container">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
</div>

.flex-container { display: flex; gap: 12px; }   /* items now sit side by side in one line */
.card           { flex: 1; }                    /* every card gets the same width */
```

CLEARER: `display: flex` makes the **container** behave like a block (full width). Its children's own `display` values (block, inline, etc.) stop mattering because flex rules take over. Use `display: inline-flex` if you want the container itself to be only as wide as its content.

Main axis and cross axis

`flex-direction`	Main axis	Cross axis
`row` (default)	Left to right	Top to bottom
`column`	Top to bottom	Left to right

The cross axis is always perpendicular to the main axis. `justify-content` works along the **main** axis; `align-items` works along the **cross** axis.

Which element gets which property?

On the PARENT (container)	On the CHILDREN (items)
`display: flex`	`flex-basis` (starting size)
`flex-direction` (row / column)	`flex-grow`, `flex-shrink`
`flex-wrap` (nowrap / wrap)	`flex` (shorthand for grow shrink basis)
`justify-content`, `align-items`, `align-content`	`order`, `align-self`
`gap`	`min-width`, `max-width`

CLEARER: Your notes already say this: a property meant for the child does nothing on the parent, and vice versa. Most "my flexbox is not working" bugs are this.

```
.container {
  display: flex;
  flex-direction: row;              /* row | column */
  flex-wrap: wrap;                  /* nowrap (default) = one line; wrap = extra items go down */
  justify-content: space-between;   /* spread along MAIN axis */
  align-items: center;              /* align along CROSS axis */
  gap: 10px;
}
.item:first-child { order: 1; }     /* default order is 0; higher numbers move later */
```

Property	Values	What it does
`justify-content`	`flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`	Distributes items along the main axis
`align-items`	`stretch` (default), `flex-start`, `center`, `flex-end`, `baseline`	Aligns items on the cross axis (one line)
`align-content`	same idea as justify-content	Spaces multiple **lines** on the cross axis. Only works when `flex-wrap: wrap` and there is more than one line

Centering anything (very common)

```
.center { display: flex; justify-content: center; align-items: center; height: 100vh; }
```

Item sizing: basis, grow, shrink

- `flex-basis` is the **starting size** along the main axis (width in a row). When set, it overrides `width`.
- `flex-grow`: how much extra space an item may take. Default `0` (do not grow).
- `flex-shrink`: whether an item may shrink when space is short. Default `1` (may shrink), `0` = never shrink.
- Shorthand: `flex: grow shrink basis`. `flex: 1` means grow 1, shrink 1, basis 0, so equal widths.

Setting	Result
`flex: 0 1 auto` (default)	Keeps natural size; shrinks only if the container is too small
`flex-grow: 1`	Item stretches to fill leftover space
`flex-shrink: 0`	Item never shrinks below its basis (may overflow the container)
`flex: 1` on all items	All items equal width
`flex: 1 1 200px` + `flex-wrap: wrap`	Items at least about 200px wide; wrap to next line when there is no space; grow to fill the row

CORRECTION: Your notes say that grow 0 and shrink 0 "behaves in a default way". Not quite: with both at 0 the item is **fixed** at its basis (100px stays 100px). The true defaults are grow 0 and shrink 1.

CLEARER: Final size rule: the item starts at `flex-basis` (if `auto`, it uses `width`, then content size). Then it grows or shrinks. Finally `min-width` and `max-width` **clamp** the result, so they always win. `max-width` limits how big it can grow; `min-width` limits how small it can shrink. This replaces your "content < width < flex-basis < max/min" chain, which reads like a ranking but is really: basis, then grow/shrink, then clamp.

11. CSS Grid (two-dimensional layout)

Grid places items in **rows and columns at the same time**, so things line up in both directions.

```
<div class="container">
  <p>1</p> <p>2</p> <p>3</p> <p>4</p>      <!-- these are grid items -->
</div>

.container {
  display: grid;
  grid-template-columns: 1fr 2fr;   /* 2 columns, widths in ratio 1:2 */
  grid-template-rows: 1fr 1fr;      /* 2 rows, equal */
  gap: 10px;                        /* space between tracks (no need for margin/padding hacks) */
}
```

By default a grid takes the full width, and its height is only as tall as the content needs. Add `height: 100vh` to make it fill the browser vertically.

Grid vocabulary

Term	Meaning
Grid container	Element with `display: grid`
Grid item	Direct children of the container
Track	A single row or a single column
Cell	Smallest unit of the grid (one row and one column crossing)
Line	The dividing lines that number the grid starting at 1 (negative numbers count from the end)
Area	Several cells merged into a rectangle that one item occupies

Sizing tracks

```
/* Fixed sizes (px or rem): do NOT adapt to screen size */
grid-template-rows: 100px 200px 400px;
grid-template-columns: 100px 200px 300px;

/* Shorthand: rows / columns separated by a slash */
grid-template: 100px 200px / 400px 300px;

/* auto: takes whatever space is left / fits the content */
grid-template-columns: 200px auto;      /* first 200px, second fills the rest of the width */
grid-template-rows: 100px auto;         /* auto row is only as tall as its content */

/* fr = fraction of the available space (most used) */
grid-template-columns: 1fr 2fr;         /* second column is twice as wide as the first */

/* repeat(count, size): avoid typing the same size again and again */
grid-template-columns: repeat(3, 200px);   /* 3 columns of 200px */
grid-template-rows:    repeat(2, 200px);   /* 2 rows of 200px   */
```

CORRECTION: In your notes, **`repeat(3, 200px)`** for columns and **`repeat(2, 200px)`** for rows is described as "8 column and 2 row". It creates **3 columns and 2 rows** (6 cells). With 4 divs, the first 3 fill row 1 and the 4th goes to row 2, column 1.

CLEARER: **`fr`** and long content: a column with a lot of content will not shrink below its content size, so **`1fr 2fr`** may not be an exact 1:2. For strictly equal columns use **`minmax(0, 1fr)`**. **`minmax(min, max)`** sets a size range: **`minmax(100px, 300px)`** is never below 100px and never above 300px.

Extra rows: grid-auto-rows

```
/* You defined 2 rows x 2 columns (4 cells) but wrote 5 divs. The 5th creates an implicit row.
   Control the height of such automatic rows like this: */
.container { grid-auto-rows: 300px; }     /* every automatically created row is 300px tall */
```

Placing items: start, end, span, area

```
.header  { grid-column-start: 1; grid-column-end: -1; }   /* from first line to last line = full width */
.header  { grid-column: 1 / -1; }                         /* same thing, shorthand */
.sidebar { grid-column: span 2; }                         /* occupy 2 columns from where it starts */

/* grid-area shorthand: row-start / column-start / row-end / column-end */
.box { grid-area: 2 / 1 / 3 / 3; }      /* starts row line 2, col line 1; ends row line 3, col line 3
                                           = 1 row high, 2 columns wide */
```

CLEARER: **Line numbers, not cell numbers.** A grid with 3 columns has 4 vertical lines (1, 2, 3, 4). **`-1`** always means the last line and **`1`** the first line. That is the "negative 1" trick in your notes. **`grid-column: 1 / 3`** covers columns 1 and 2.

- Like flexbox, grid items also have an **`order`** property (default **`0`**).
- Use **`gap`** between tracks instead of padding or margin on the items.
- **Grid items can overlap** (give them the same cell and set **`z-index`**). Flexbox items cannot overlap.

BONUS: Grid with fixed pixels is not responsive. The classic fully responsive grid with no media queries: **`grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));`**. It fits as many 200px+ columns as the screen allows.

12. Flexbox vs Grid

Flexbox	Grid
One dimension (a row **or** a column)	Two dimensions (rows **and** columns)
Content decides the size (content-first)	Layout decides the size (layout-first)
Best for navbars, button groups, centering, card rows	Best for page layouts, galleries, dashboards
Items are hard to line up in both directions	Items line up in both rows and columns
Items cannot overlap	Items can overlap

Most real pages use **both**: grid for the overall page, flexbox inside the pieces, and sometimes the reverse.

```
.page   { display: grid; grid-template-columns: 250px 1fr; gap: 16px; }   /* sidebar + content */
.navbar { display: flex; justify-content: space-between; align-items: center; }   /* inside header */
```

13. Bootstrap

- A CSS framework: ready-made CSS (and JS) with classes for layout, buttons, cards, navbars, modals, etc. Bootstrap is the most used one.
- **Pros:** fast, lots of pre-built components, responsive by default.
- **Cons:** many sites look alike, you must learn its class names, and it loads extra CSS you may not use.

Setup using a CDN (Content Delivery Network)

Two links are needed: the CSS for the styling, and the JS for functionality such as dropdowns, menus and modals.

```
<head>
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet" href="./style.css">   <!-- YOUR css goes AFTER bootstrap to override it -->
</head>
<body>
  ...
  <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
```

BONUS: Check getbootstrap.com for the latest version number and copy the exact links from its "Quick start" page. The JS link goes just before `</body>`.

The 12-column layout

Three parts, nested: **container** then **row** then **columns**. The columns in one row should add up to 12; extra columns wrap to the next line.

```
<div class="container">
  <div class="row">
    <div class="col-2">col-2</div>     <!-- 2 / 12 of the width -->
    <div class="col-4">col-4</div>     <!-- 4 / 12 -->
    <div class="col-6">col-6</div>     <!-- 6 / 12   (2 + 4 + 6 = 12) -->
  </div>
</div>
```

Breakpoints (no media queries needed)

Prefix	Screen width	Device (roughly)
(none) `col-*`	all sizes	everything
`col-sm-*`	576px and above	large phones
`col-md-*`	768px and above	tablets
`col-lg-*`	992px and above	laptops
`col-xl-*`	1200px and above	desktops

```
<!-- 1 column on phones, 2 on tablets, 3 on laptops -->
<div class="row">
  <div class="col-12 col-md-6 col-lg-4">Card</div>
  <div class="col-12 col-md-6 col-lg-4">Card</div>
  <div class="col-12 col-md-6 col-lg-4">Card</div>
</div>
```

CLEARER: Your notes say screens below 576px take full width. That happens when you use the prefixed classes (`col-sm-*`, `col-md-*`, ...): below that breakpoint the column becomes full width (like `col-12`). A plain `col-6` has no breakpoint, so it stays 50% even on phones. You choose the number per screen size.

BONUS: To override Bootstrap, put your own CSS link below the Bootstrap link and be specific (e.g. `.card.my-card`) or put the rules in a `<style>` block after the link.

14. Quick revision

One-line cheat sheet

Topic	Remember
Selectors	`h1` element, `.c` class (many), `#i` id (one), `[a]` attribute, `*` all
Combinators	`,` group | `>` direct child | space descendant | no space same element
Cascade	`!important` > specificity (inline > id > class > element) > later rule
Units	`rem` = root, `em` = parent, `vh` = viewport height
Box model	content, padding, border, margin; shorthand clockwise T R B L
Display	block = full width, inline = no size, inline-block = both, none = gone
Position	parent `relative` + child `absolute`; `fixed` = viewport; `sticky` = sticks after scrolling
Media query	`max-width` = up to, `min-width` = from
Flex	`display:flex`; `justify-content` = main axis; `align-items` = cross axis; `flex: 1` = equal
Grid	`grid-template-columns: repeat(3, 1fr)`; `gap`; `grid-column: 1 / -1`
Bootstrap	container > row > col; CSS link in head, JS before `</body>`

Common mistakes

- Forgetting `.` or `#` in the selector: `card { }` targets a tag called card, not the class.
- Setting `width`/`height` on an inline element (like `span`): nothing happens. Use `inline-block` or `block`.
- Using `position: absolute` without `position: relative` on the parent: the element flies to the page corner.
- Putting `flex-direction` on a child or `flex-basis` on the parent: has no effect.
- Using `align-content` without `flex-wrap: wrap`: has no effect (use `align-items` instead).
- Using `float` to build page layouts: use flexbox or grid.
- Linking your CSS before Bootstrap and wondering why your styles lose.
- Forgetting the viewport meta tag, so media queries seem broken on a real phone.

What was changed from the handwritten notes

Topic	Handwritten note	Fixed version
Cascade order	Importance > Type > Specificity > Position; external weakest	!important > specificity > order. Internal and external have equal weight
Chaining selector	Always starts with an element	Only when an element name is used; `.a.b` is valid
Flex shrink/grow	Both 0 = default behaviour	Both 0 = fixed size. Defaults are grow 0, shrink 1
Flex size order	content < width < basis < max/min	basis, then grow/shrink, then clamped by min/max
Grid repeat	`repeat(3, 200px)` makes 8 columns	It makes 3 columns
Inline elements	`p` and `img` listed as inline	`p` is block; `img` is inline but accepts width/height